Swaraj Patil **D20A/42**

Blockchain Experiment No 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:
A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
- Transparency: The contract code and transactions are visible on the blockchain.
- Security: Once deployed, contracts are immutable and tamper-proof.
- Cost-effective: Removes intermediaries and reduces transaction costs.
- Trustless System: Participants can interact without needing to trust each other directly. Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:

Contract 1:VotingSystem

Voting System :

```
// SPDX-License-Identifier: MIT pragma
solidity ^0.8.0;
contract VotingSystem {

   struct Candidate

{        uint id;
string name;
uint voteCount;
   }

   mapping(uint => Candidate) public candidates;
mapping(address => bool) public voters;

   uint public candidatesCount;
```

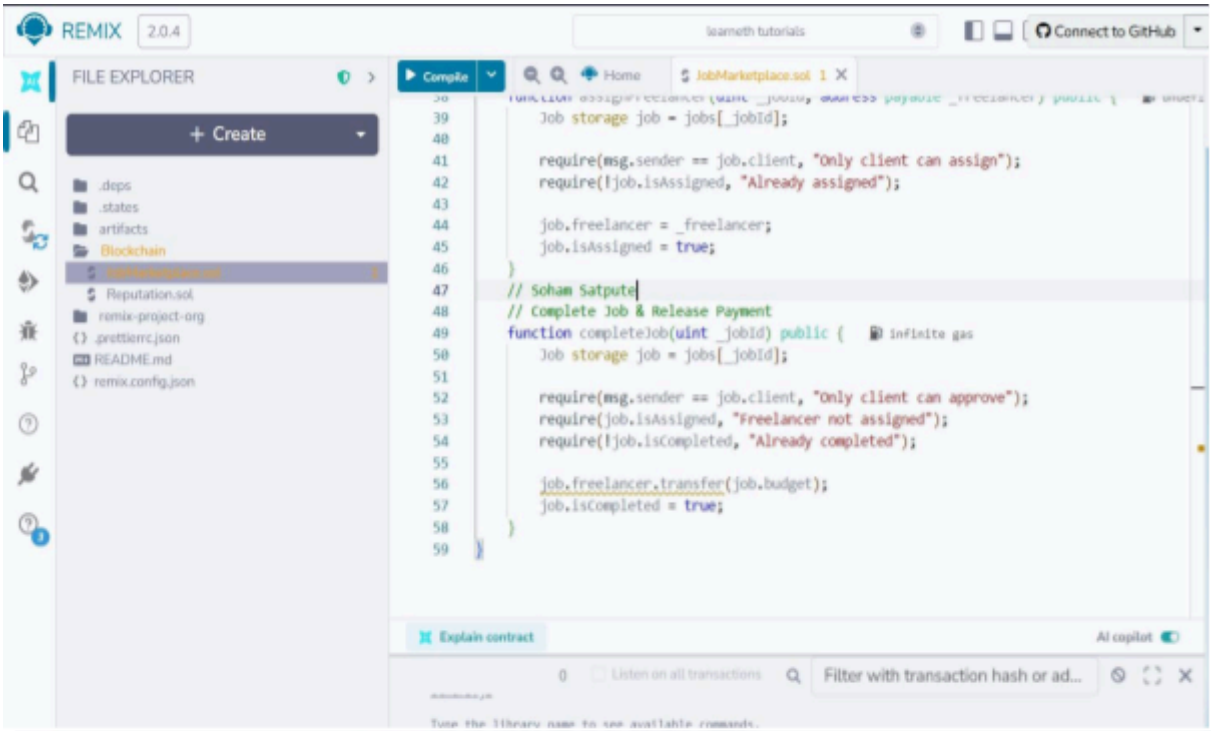
```
 function addCandidate(string memory _name) public {
candidatesCount++;    candidates[candidatesCount]    =
Candidate(candidatesCount, _name, 0);
   }

   function vote(uint _candidateId) public { require(!voters[msg.sender], "Already
voted!");  require(_candidateId > 0 && _candidateId <= candidatesCount, "Invalid
candidate");

      voters[msg.sender] = true;
candidates[_candidateId].voteCount++;
   }

   function getVotes(uint _candidateId) public view returns (uint) {
return candidates[_candidateId].voteCount;
   }
}
```

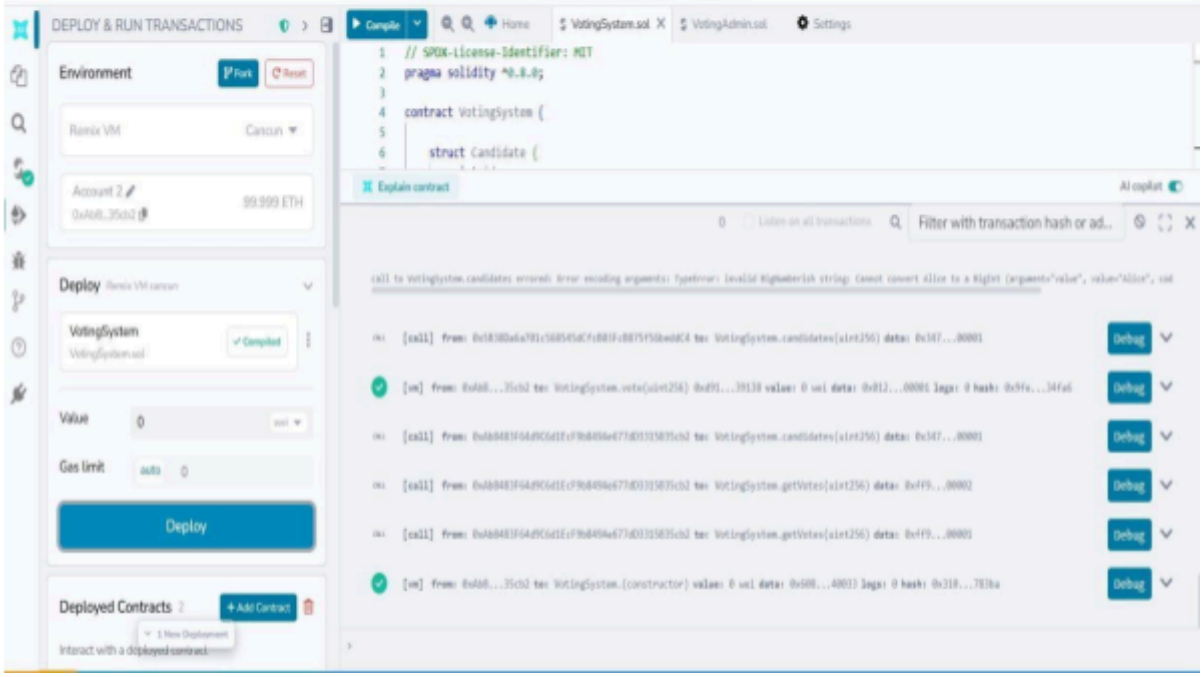
Contract 2:

```
   Voting Admin;
   // SPDX-License-Identifier: MIT pragma
   solidity ^0.8.0;

   contract VotingAdmin {

      address public admin;
      bool public votingActive;

      constructor() {
   admin = msg.sender;
         votingActive = false;
      }

      modifier onlyAdmin() {
   require(msg.sender == admin, "Not admin");
         _;
      }

      function startVoting() public onlyAdmin {
         votingActive = true;
      }

      function endVoting() public onlyAdmin {
         votingActive = false;    }     function
   getStatus() public view returns (bool) {
         return votingActive;
      }
   }
```

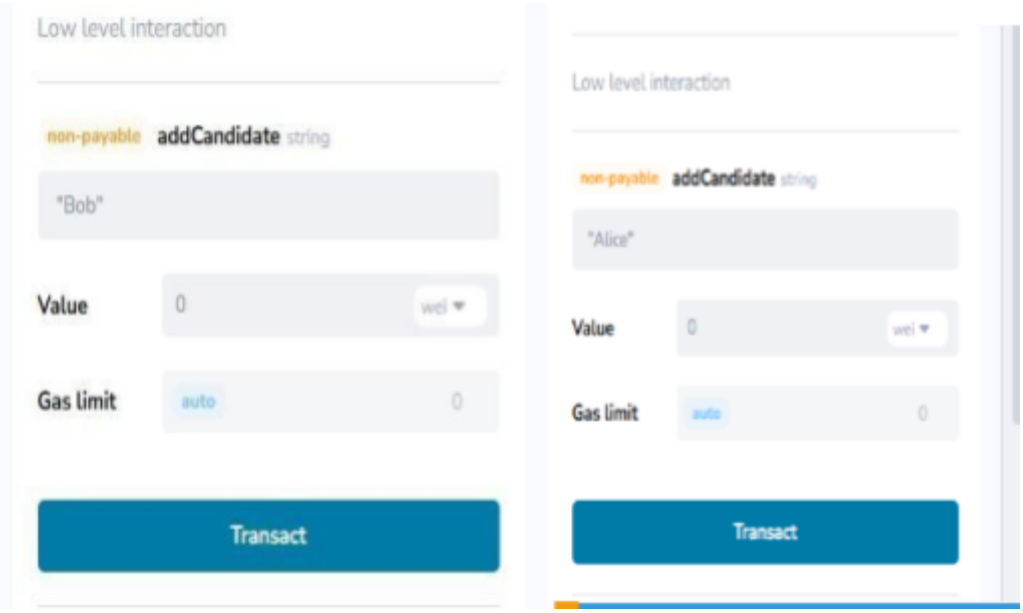
Implementation :

ScreenShots :

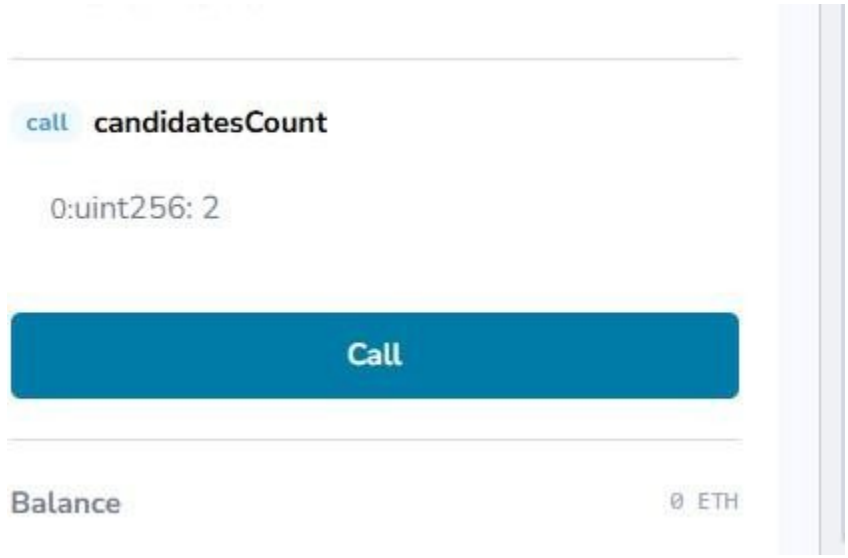




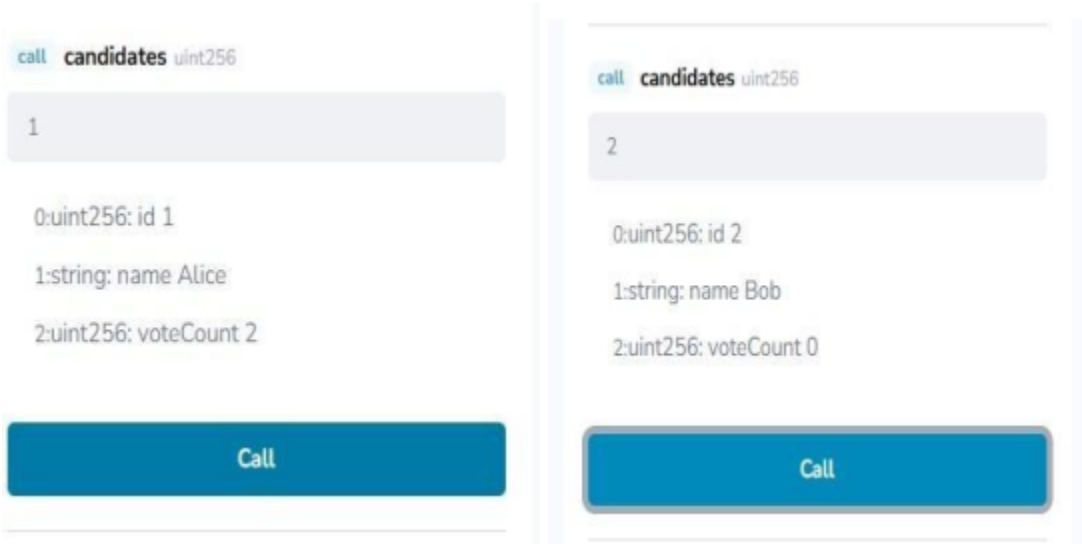
Transaction deployed successfully



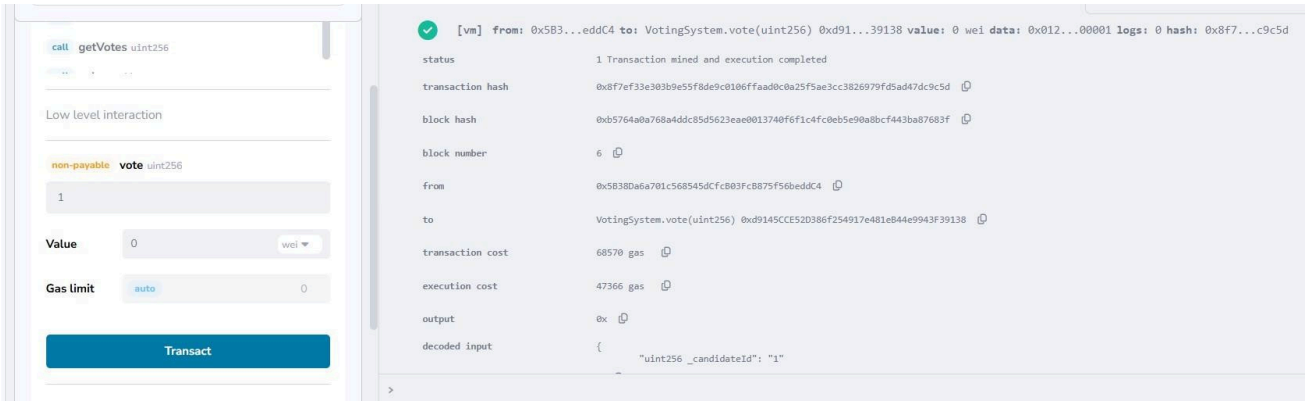
Both Candidates Added



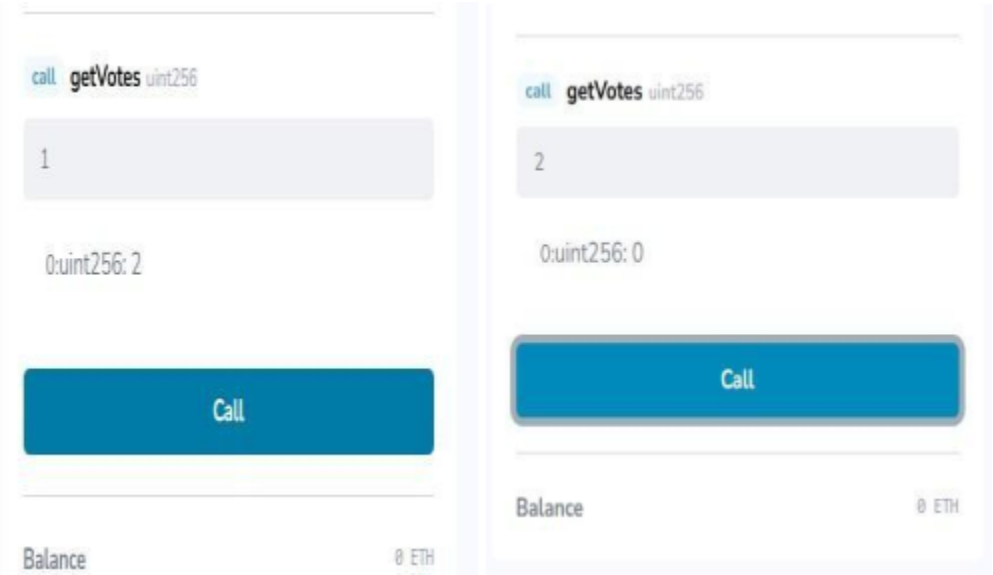
Candidate Count



Candidates



Vote



Vote Count

Conclusion:

The Blockchain-based Voting System was successfully implemented using Solidity smart contracts and deployed on the Remix IDE. Two smart contracts were designed to handle voting operations and administrative control, ensuring modularity and better system design.

The system allowed adding candidates, casting votes, and retrieving vote counts while preventing double voting through the use of mappings and conditions. All transactions were executed on the blockchain, ensuring transparency, security, and immutability of data.

Thus, the experiment helped in understanding the working of smart contracts, their deployment, and execution in a decentralized environment. The objectives of designing and implementing smart contracts were successfully achieved.